

Lecture 05

2026-04-14

Table of contents

Welcome Back	2
Anonymous Activity 1	2
QR Code (it's getting better)	2
Answers to questions I have received	3
tables	3
load()	3
save()	4
Where?	4
Example	4
Later	5
DataCamp	6
Closing thought from last time	6
What is a low level language? Assembly v R	6
For your consideration	7
A final thought on this	7
Attendance Opportunity	8
Activity #2: Please Greet Your Neighbor(s)	8
Data Week	8
What are they looking for?	8
Depends, but	8
What to focus on this week	9
Recall: Workflow (in research)	9
Recall: Workflow (in workplaces)	10
It's only data...	10
Read into R	10

Activity 3 - Running an R Script	11
Code	11
Saving your workspace	11
Excel and Others	11
Still Others	12
Who are they?	12
What happens when data is not perfectly read?	12
What is NA in R?	13
What does NA mean to R?	14
What happens when data is not perfect?	14
What happens when data is not perfect (cont'd)	15
How do we remove a row (or rows)?	16
Renumbering your row names (extra)	17
Takeaways	18
Questions?	18
Data is Everywhere	18
What is JSON?	19
Simple Example	19
Code & Result	19
Practical Example	22
Result	22
Interpretation	24
Bigger Challenges - Multiple files	24
For your consideration	24
Takeaways	25
Thank You	25

Welcome Back

Anonymous Activity 1

QR Code (it's getting better)

This is anonymous & optional, but helpful to Stats 20,



or here is a [direct link](#)

Answers to questions I have received

tables

- If you like pretty tables, there are several libraries that you can utilize.
- [kableExtra](#)
- [gt](#) written by the folks at RStudio
- [dt](#) typically use with dashboards or online
- You can make reasonably nice tables in Quarto

load()

- Yes, you should know how to load a saved R workspace and you should probably learn how to save one though I won't test you on it (others might):

```
## Assumes you have a .RData file with this name in your project folder (AKA current working  
load("spotify_songs.RData")  
ls() # a check to make sure something loaded
```

```
[1] "spotify_songs"
```

- The R objects stored inside of a saved R workspace have no necessary relationship to the external file name of the workspace. Hence `ls()` to check.

save()

- Yes, you can save a workspace, if you want to be lazy, just issue a `save.image()` at end of your R session and it will save everything you created, the good and the not so good:

```
save.image()
```

- IMO is better to have a more controlled/mindful `save()`

```
x <- runif(20)
y <- c(1, TRUE, "FALSE")
save(x, y, file = "junk.RData")
```

Where?

- Where R objects are saved can be confusing and is valuable to understand if you plan to perform more computation in an upper division course (e.g., 102 series) in the future.

```
getwd() # what is the current working directory/folder
```

```
[1] "/Users/vivian/Desktop/SP26Stats20/website/lectures/Lecture0506"
```

```
list.files() # what is in the working directory
```

```
[1] "_extensions"      "images"            "Lecture05_files"
[4] "Lecture05.qmd"     "Lecture05.rmarkdown" "my_folder"
[7] "spotify_songs.RData" "starwars.RData"
```

- What is important is to know how to tell R how to find something, like a data file or an image file. Or to validate that a file is there. Or to create one if its not.
- I don't ask you to do that, but if you want practice, create a sub-directory or a subfolder on your computer drive and put something in it, like an image file or an R Data file
- Then try to read it into R

Example

- How do we create a folder from within R and why would we do this?
- You could store output in an orderly manner knowing how to do this

```
x <- seq(from = 1, to = 1000, by = 2)
y <- rnorm(n = 160, mean = 120, sd = 15)

dir.create("my_folder") # if it already exists it will throw a FALSE but not an error
```

Warning in dir.create("my_folder"): 'my_folder' already exists

```
save(x, y, file = "my_folder/xy.RData") # Save the file "xy.RData" in the "my_folder"
```

Later

- For those of you who are just dying in here because I move too slow, eventually I will want you to be able to read and understand this also possibly write something similar (esp. for 102):

```
# Check if the folder "my_folder" exists, if not, create it.
if (!dir.exists("my_folder")) {
  dir.create("my_folder")
}
# Save the file "xy.RData" in the "my_folder" if it does not exist
# otherwise (else) tell me it is already there.
if (!file.exists("my_folder/xy.RData")) {
  save(x, y, file = "my_folder/xy.RData")
  cat("The file was created and saved successfully.\n")
} else {
  cat("The file already exists.\n")
}
```

The file already exists.

```
## prove it to me
{cat("These are the files in the folder:\n")
files_in_folder <- list.files("my_folder")
files_in_folder}
```

These are the files in the folder:

```
[1] "xy.RData"
```

- We will visit flow control then functions after we look at data and visualization

DataCamp

- I don't assign homework, I think projects are more realistic
- But DataCamp is there to give you extra practice. I curate their "chapters" as best as I can to match the subjects in lecture
- DataCamp is structured to let you do things at your own pace
- So I just need to see that you attempted the assignment
- You do not need to complete everything and you do not need to get it 100% correct
- The deadlines are there so you can pace yourselves, I will only calculate completion at the end of the quarter
- The Intro to Stats module is there for the kind students who let me know that they have forgotten a lot of their intro stats training. I appreciate honesty.

Closing thought from last time

What is a low level language? Assembly v R

- Assembly invented in 1947, R 1993, fyi: Python 1989

```
.model small
.stack 100h

.data
a dw 2
b dw 5

.code
main proc
    mov ax, @data
    mov ds, ax

    mov ax, a
    mov bx, b
    add ax, bx

    add al, 30h
    mov dl, al
    mov ah, 02h
    int 21h

    mov ah, 4Ch
```

```
    int 21h
main endp
end main
```

- In R

```
a <- 2
b <- 5
```

```
a + b
```

- Extra (binary, the OG), 2 and 5, represented, not added yet

```
00000000000000010
00000000000000101
```

- And the representation for 7 may make sense to you

```
00000000000000111
```

For your consideration

- In assembly, we must tell the computer every tiny step. Assembly required humans to think/communicate like machines.
- In R, Python, and other modern languages, we write what we want in a compact and more natural form
- Programming languages evolved to let humans think less about machine details and more about ideas, data, and problems.
- LLMs are part of the same long arc of evolution. They allow us to express intent in ordinary language. So “please add 2 and 5” as in the previous slide
- Assembly -> C -> Python -> LLMs(?) older languages were not completely replaced – but their application may be narrower

A final thought on this

- If you ask an LLM (I asked ChatGPT) what languages it rests upon, perhaps this answer is not too surprising:

Human language

↓

Python (or similar high-level language)

↓

C / C++ (fast execution)

↓

Assembly

↓

Machine code (binary)

- it relies on many different tools for many different applications

Attendance Opportunity

Activity #2: Please Greet Your Neighbor(s)

- You are experts at greeting your neighbors now.
- You need at least one different neighbor each week for your attendance credit (only one photo required)
- You cannot receive credit if it's only you in the photo
- [Social Connection has value](#)
- Meet a new classmate, take a photograph.

Data Week

What are they looking for?

- [Red Bull Internship](#)
- [Booz Allen Hamilton](#)
- [TikTok](#)
- [First American \(Title\)](#)
- [Petco](#)

Depends, but

- Can you work with real data (messy, large, multiple sources)
- Can you write a script (or more advanced, model outcomes)
- Can you communicate your findings (present, tell a story, connect)
- Do you have the an understanding of the business context?

What to focus on this week

- Working with real data
- Knowing how to “data wrangle” – basic making messy data suitable for analysis
 - Handling missing values
 - Converting data types
 - Reshaping data (going from wide format to long and back)
 - Joining data
 - Cleaning text or categories

Recall: Workflow (in research)

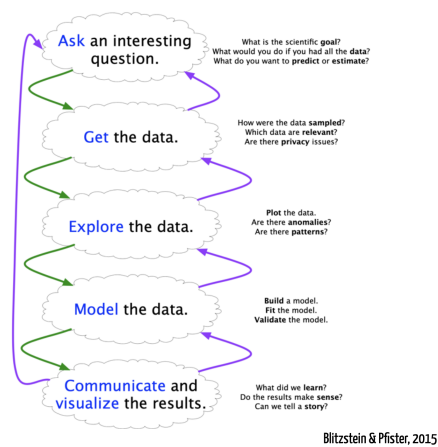


Figure 1: A Data Science Workflow

Recall: Workflow (in workplaces)

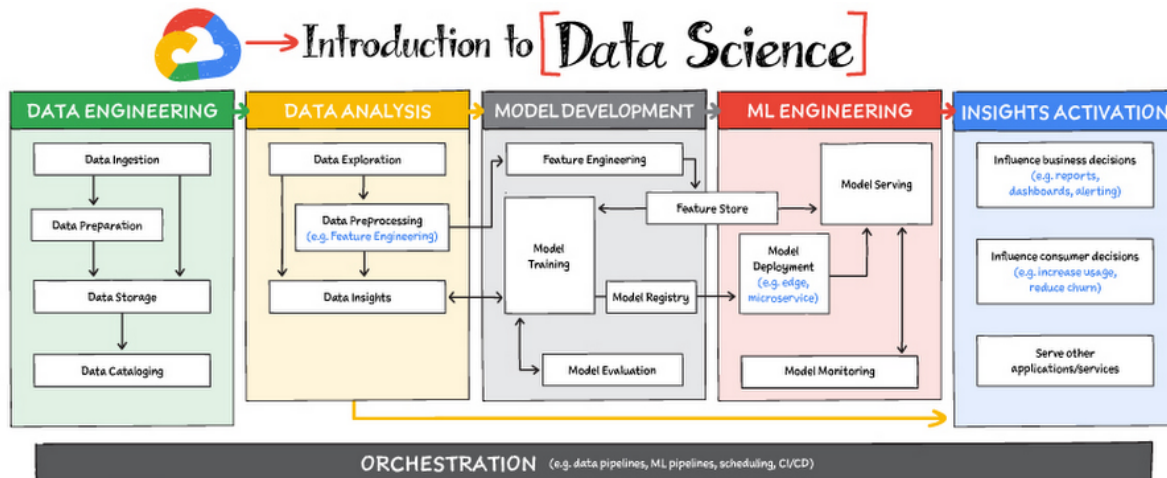


Figure 2: From Google

It's only data...

- The content of all four of these are exactly the same (your Project 1 dataset), the formatting is different:

comma separated values (.csv)

tab separated values (.tsv)

white space delimited values (.txt)

json example – extra

Read into R

- We do not need to load any special libraries/install packages for the first three. We will read the fourth one later.
- We will use base R (core R, what is available at installation) and the functions `read.csv()`, `read.delim()`, and `read.table()`
- These are relatively simple to read because they are DELIMITED (have a separator) between variables (AKA fields)

- There is a logical workflow (1) need to know their location (2) need a function to convert the data into an R data frame (3A) need to check my work - did they get read (3B) are all 3 the same?

Activity 3 - Running an R Script

Code

- Please turn to RStudio
- We are going to work through an R script named **remote_delimited.R** you can find it in Week 3 under Tuesday
- Read data off the web and save data to your to your drive
- If you don't have RStudio today, you can copy and paste most of this to Web-R
- But it is nice to have a friend so you don't get lost because the Web-R will have some problems.

Saving your workspace

- You should know how to save your work, **save()** works just fine, here is a multiple object example:

```
save(pokemon_data_csv, pokemon_data_tsv, pokemon_data_txt, file = "all_my_pokemon.RData")
```

- When you **load("all_my_pokemon.RData")** in another session, all 3 data frames will be restored to your global environment.

Excel and Others

- There are special libraries that we can use to read Excel files (later). You could also convert your Excel file to .csv for example
- But we can read Google sheets using R as long as we own them or have been given permission to read them:

```
in_n_out_locs <- read.csv("https://docs.google.com/spreadsheets/d/1eTkMGK4Bsab5cN-UtQgqdosGD")
in_n_out_latlon <- read.csv("https://docs.google.com/spreadsheets/d/1eTkMGK4Bsab5cN-UtQgqdosGD")
names(in_n_out_locs)
```

```
[1] "name"      "address"  "city"     "state"    "zip"
```

```
names(in_n_out_latlon)
```

```
[1] "name"      "zip"       "latitude"  "longitude"
```

Still Others

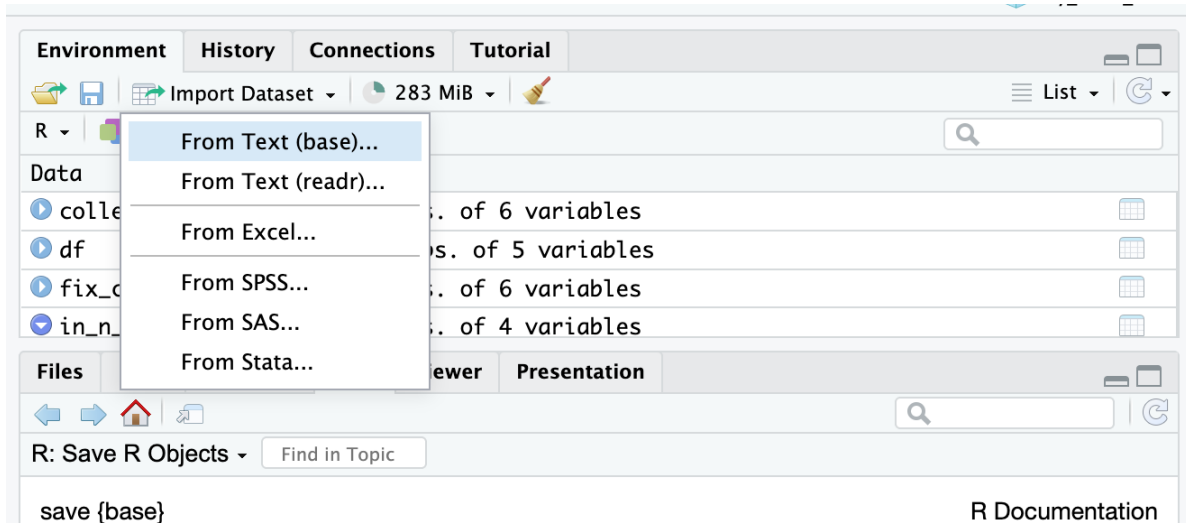


Figure 3: some competitors

Who are they?

- These are specialized statistical packages
- They almost all have “bridges” to Python and/or R.
- SAS 1976 (pharma); SPSS 1968 (psychologists); Stata 1985 (economists)
- I won’t ask you about these. I am OK if you point and click your way to a solution or ask AI.

What happens when data is not perfectly read?

- It is getting more difficult to show you an example as computers don’t usually make mistakes in collecting/constructing data, but people do:

```
colleges <- read.delim("https://raw.githubusercontent.com/leww/data-for-example/refs/heads/main/colleges.csv")
```

	Name	Quality	Academics	Sports	Cost	Overall
1	Princeton	1	3	11	75	1
2	UNC	46	22	10	31	2
3	Stanford	4	5	1	110	3
4	Yale	14	4	5	100	4
5	Virginia	24	30	29	41	5
6	Williams	2	25	19	80	6
7	Michigan	38	24	9	64	7
8	NotreDame	6	31	32	73	8
9	California	50	15	20	61	9
10	Harvard	12	1	39	98	10
11	UCLA	67	20	2	66	11
12	Columbia	33	9	35	79	12
13	USNA	28	88	45	1	13
14	Duke	5	10	48	104	14
15	Amherst	3	28	50	88	15
16	Texas	106	36	12	18	16
17	Florida	71	38	59	7	17
18	Rice	19	11	89	58	18
19	Dartmouth	11	14	52	103	19
20	Middlebury	21	40	15	108	20
21	NewJersey	44	102	14	25	21
22	PennState	69	56	17	44	22
23	Oklahoma	77	86	22	9	23
24	CalTech	NA	17	6	106	65
25	23	NA	NA	NA	NA	NA
26	Maryland	91	46	28	34	25
27	LSU	75	99	13	12	25

- Notice no warning. You should always check a data read.

What is NA in R?

- NA represents unknown or unavailable data
- It is NOT positive
- It is NOT negative
- It is not zero
- not a number at all

There should be a value here, but we don't know what it is.

What does NA mean to R?

```
5 + NA # NA
```

```
[1] NA
```

```
NA == 5 # NA
```

```
[1] NA
```

```
NA == NA # NA
```

```
[1] NA
```

```
is.na(NA) # TRUE
```

```
[1] TRUE
```

- By not substituting a value, R lets the user decide how to address NA

What happens when data is not perfect?

- With a tiny dataset like this, it's easy to find the problem

← → ↻ raw.githubusercontent.com/lewv/data-for-example/refs/heads/main/colleges.txt

Name	Quality	Academics	Sports	Cost	Overall
Princeton	1	3	11	75	1
UNC	46	22	10	31	2
Stanford	4	5	1	110	3
Yale	14	4	5	100	4
Virginia	24	30	29	41	5
Williams	2	25	19	80	6
Michigan	38	24	9	64	7
NotreDame	6	31	32	73	8
California	50	15	20	61	9
Harvard	12	1	39	98	10
UCLA	67	20	2	66	11
Columbia	33	9	35	79	12
USNA	28	88	45	1	13
Duke	5	10	48	104	14
Amherst	3	28	50	88	15
Texas	106	36	12	18	16
Florida	71	38	59	7	17
Rice	19	11	89	58	18
Dartmouth	11	14	52	103	19
Middlebury	21	40	15	108	20
NewJersey	44	102	14	25	21
PennState	69	56	17	44	22
Oklahoma	77	86	22	9	23
CalTech	17	6	106	65	23
Maryland	91	46	28	34	25
LSU	75	99	13	12	25

Figure 4: dataset with mistakes

What happens when data is not perfect (cont'd)

- apply correction, typically by substitution (but could drop the observation)
- but usually by filtering/subsetting (next lecture) first though locate the error
- in our tiny example, we are going to correct the two rows that are incorrect
- we haven't gotten to all of the details here, but we will

```
colleges[24, 2:6] # has a mistake
```

```
Quality Academics Sports Cost Overall
24      NA         17      6  106      65
```

```
colleges[24, 2:6] <- c(17, 6, 106, 65, 23) # overwrite
colleges # check
```

```
Name Quality Academics Sports Cost Overall
```

1	Princeton	1	3	11	75	1
2	UNC	46	22	10	31	2
3	Stanford	4	5	1	110	3
4	Yale	14	4	5	100	4
5	Virginia	24	30	29	41	5
6	Williams	2	25	19	80	6
7	Michigan	38	24	9	64	7
8	NotreDame	6	31	32	73	8
9	California	50	15	20	61	9
10	Harvard	12	1	39	98	10
11	UCLA	67	20	2	66	11
12	Columbia	33	9	35	79	12
13	USNA	28	88	45	1	13
14	Duke	5	10	48	104	14
15	Amherst	3	28	50	88	15
16	Texas	106	36	12	18	16
17	Florida	71	38	59	7	17
18	Rice	19	11	89	58	18
19	Dartmouth	11	14	52	103	19
20	Middlebury	21	40	15	108	20
21	NewJersey	44	102	14	25	21
22	PennState	69	56	17	44	22
23	Oklahoma	77	86	22	9	23
24	CalTech	17	6	106	65	23
25	23	NA	NA	NA	NA	NA
26	Maryland	91	46	28	34	25
27	LSU	75	99	13	12	25

How do we remove a row (or rows)?

- usually in a larger dataset, we apply a condition
- here, we are just going to drop the one row

```
fix_colleges <- colleges[-25, ] # copy the original, don't clobber it
fix_colleges # the numbering is off
```

	Name	Quality	Academics	Sports	Cost	Overall
1	Princeton	1	3	11	75	1
2	UNC	46	22	10	31	2
3	Stanford	4	5	1	110	3
4	Yale	14	4	5	100	4

5	Virginia	24	30	29	41	5
6	Williams	2	25	19	80	6
7	Michigan	38	24	9	64	7
8	NotreDame	6	31	32	73	8
9	California	50	15	20	61	9
10	Harvard	12	1	39	98	10
11	UCLA	67	20	2	66	11
12	Columbia	33	9	35	79	12
13	USNA	28	88	45	1	13
14	Duke	5	10	48	104	14
15	Amherst	3	28	50	88	15
16	Texas	106	36	12	18	16
17	Florida	71	38	59	7	17
18	Rice	19	11	89	58	18
19	Dartmouth	11	14	52	103	19
20	Middlebury	21	40	15	108	20
21	NewJersey	44	102	14	25	21
22	PennState	69	56	17	44	22
23	Oklahoma	77	86	22	9	23
24	CalTech	17	6	106	65	23
26	Maryland	91	46	28	34	25
27	LSU	75	99	13	12	25

Renumbering your row names (extra)

- if you must... this is more sequence generating:

```
rownames(fix_colleges) # we lost row 25
```

```
[1] "1" "2" "3" "4" "5" "6" "7" "8" "9" "10" "11" "12" "13" "14" "15"
[16] "16" "17" "18" "19" "20" "21" "22" "23" "24" "26" "27"
```

```
rownames(fix_colleges) <- 1:26 # numeric vector
rownames(fix_colleges) # coerced to character
```

```
[1] "1" "2" "3" "4" "5" "6" "7" "8" "9" "10" "11" "12" "13" "14" "15"
[16] "16" "17" "18" "19" "20" "21" "22" "23" "24" "25" "26"
```

```
fix_colleges
```

	Name	Quality	Academics	Sports	Cost	Overall
1	Princeton	1	3	11	75	1
2	UNC	46	22	10	31	2
3	Stanford	4	5	1	110	3
4	Yale	14	4	5	100	4
5	Virginia	24	30	29	41	5
6	Williams	2	25	19	80	6
7	Michigan	38	24	9	64	7
8	NotreDame	6	31	32	73	8
9	California	50	15	20	61	9
10	Harvard	12	1	39	98	10
11	UCLA	67	20	2	66	11
12	Columbia	33	9	35	79	12
13	USNA	28	88	45	1	13
14	Duke	5	10	48	104	14
15	Amherst	3	28	50	88	15
16	Texas	106	36	12	18	16
17	Florida	71	38	59	7	17
18	Rice	19	11	89	58	18
19	Dartmouth	11	14	52	103	19
20	Middlebury	21	40	15	108	20
21	NewJersey	44	102	14	25	21
22	PennState	69	56	17	44	22
23	Oklahoma	77	86	22	9	23
24	CalTech	17	6	106	65	23
25	Maryland	91	46	28	34	25
26	LSU	75	99	13	12	25

Takeaways

- Data can be imperfect; you might not be warned
- There are multiple solutions; weigh the pros and cons
- The corrections gave no feedback; therefore always check your results
- Small datasets like this example are easy; we must be clever about finding mistakes and correcting them in larger ones

Questions?

Data is Everywhere

- If you would turn to Week 3 Module, Tuesday

- Kaggle
- UCI Machine Learning Repository
- etc.

What is JSON?

- JSON stands for JavaScript Object Notation
- JSON is a data-interchange format
- JSON is language independent text so it can be used by different software
- JSON is “human readable” and easy to understand
- JSON structure is (or can be) hierarchical (values within values)
- JSON is short and easy to write
- JSON syntax is a subset of the JavaScript syntax.
- The file type for JSON files is “.json”

Simple Example

- This is not meant to be definitive exploration of json files.
- I like the jsonlite package mostly because I know the maintainer personally.
- First, install then activate the jsonlite library. `library(jsonlite)` and you will need `library(httr)` for my second example to work
- But try the first example with a fromJSON before moving to more complex forms

Code & Result

```
library(jsonlite)

url_films <- "https://gist.githubusercontent.com/saniyusuf/406b843afdfb9c6a86e25753fe2761f4/1"

films <- fromJSON(url_films)

head(films)
```

	Title	Year	Rated	Released	Runtime
1	Avatar	2009	PG-13	18 Dec 2009	162 min
2	I Am Legend	2007	PG-13	14 Dec 2007	101 min
3	300	2006	R	09 Mar 2007	117 min
4	The Avengers	2012	PG-13	04 May 2012	143 min

5 The Wolf of Wall Street 2013 R 25 Dec 2013 180 min
6 Interstellar 2014 PG-13 07 Nov 2014 169 min

	Genre	Director
1	Action, Adventure, Fantasy	James Cameron
2	Drama, Horror, Sci-Fi	Francis Lawrence
3	Action, Drama, Fantasy	Zack Snyder
4	Action, Sci-Fi, Thriller	Joss Whedon
5	Biography, Comedy, Crime	Martin Scorsese
6	Adventure, Drama, Sci-Fi	Christopher Nolan

1
2 Mark Protosevich (screenplay), Akiva Goldsman (screenplay), Richard Matheson (novel)
3 Zack Snyder (screenplay), Kurt Johnstad (screenplay), Michael Gordon (screenplay), Frank M.
4 Joss Whedon (screenplay)
5
6

Actors

1 Sam Worthington, Zoe Saldana, Sigourney Weaver, Stephen Lang
2 Will Smith, Alice Braga, Charlie Tahan, Salli Richardson-Whitfield
3 Gerard Butler, Lena Headey, Dominic West, David Wenham
4 Robert Downey Jr., Chris Evans, Mark Ruffalo, Chris Hemsworth
5 Leonardo DiCaprio, Jonah Hill, Margot Robbie, Matthew McConaughey
6 Ellen Burstyn, Matthew McConaughey, Mackenzie Foy, John Lithgow

1 A paraplegic marine dispatched to the moon Pandora on a unique mission becomes
2 Years after a plague kills most of humanity and transforms the rest into
3 King Leonidas.
4 Earth's mightiest heroes must come together and learn to fight as a team if t
5 Based on the true story of Jordan Belfort, from his rise to a wealthy stock-
broker living the high life to his fall involving crime, corruption and the federal government.
6 A team of explorers

	Language	Country
1	English, Spanish	USA, UK
2	English	USA
3	English	USA
4	English, Russian	USA
5	English, French	USA
6	English	USA, UK

Awards

1 Won 3 Oscars. Another 80 wins & 121 nominations.
2 9 wins & 21 nominations.
3 16 wins & 42 nominations.
4 Nominated for 1 Oscar. Another 34 wins & 75 nominations.


```

na.ssl-images-amazon.com/images/M/MV5BMTg4Njk4MzY0N15BM15BanBnXkFtZTgwMzIyODgxMzE@._V1_SX150
na.ssl-images-amazon.com/images/M/MV5BMzE3MTM0MTc3M15BM15BanBnXkFtZTgwMDIyODgxMzE@._V1_SX150
na.ssl-images-amazon.com/images/M/MV5BNjYzNjE2NDk3N15BM15BanBnXkFtZTgwNzEyODgxMzE@._V1_SX150
totalSeasons ComingSoon
1          <NA>      NA
2          <NA>      NA
3          <NA>      NA
4          <NA>      NA
5          <NA>      NA
6          <NA>      NA

```

Practical Example

- [SF Film Locations](#)

```

library(httr) # need this to get the URL
library(jsonlite) # need this to convert the json to a data frame

url_sf <- "https://data.sfgov.org/resource/yitu-d5am.json"

# first, make sure the URL is valid
check <- GET(url_sf, verbose())
status_code(check) # should return 200

```

```
[1] 200
```

```

# we see there are limits imposed, only 1000
locations_check <- fromJSON(content(check, "text"), flatten = TRUE)

```

Result

```
head(locations_check)
```

	title	release_year	locations
1	Milk	2008	El Camino Del Mar
2	Chance Season 2	2017	Coit Tower
3	Red Widow	2013	Mason & Sacramento St
4	A Taiwanese Tale of Two Cities	2018	Pier 5
5	Americana	2015	Palace of Fine Arts

6	Desperate Measures		1998	<NA>	
	production_company			distributor	
1	Focus Features			Focus Features	
2	TVM Productions Inc.			Hulu	
3	Beyond Pix		American Broadcasting Company (ABC)		
4	Envision Productions Inc.			Netflix	
5	Sutro Films LLC			<NA>	
6	Mandalay Entertainment			TriStar Pictures	
	director			writer	
1	Gus Van Sant			Dustin Lance Black	
2	Rozann Dawson			Alexandra Cunningham	
3	Alon Aranya			Melissa Rosenberg	
4	Tien-Lun Yeh Ling-Hui Chen, Nancy Chen, Chih-Chi Fan, Chia-Hui Lin				
5	Zachary Shedd			Zachary Shedd	
6	Barbet Schroeder			David Klass	
	actor_1	actor_2	longitude	latitude	analysis_neighborhood
1	Sean Penn	Emile Hirsch	-122.4962354	37.7857806	Lincoln Park
2	Hugh Laurie	Greta Lee	-122.4058222	37.8023949	North Beach
3	Radha Mitchell	Sterling Beaumon	-122.4110311	37.7927806	Nob Hill
4	Tammy Cheng	Peggy Tseng	-122.3960343	37.7980698	<NA>
5	Kelli Garner	Jack Davenport	-122.4484231	37.8029308	Marina
6	Michael Keaton	Andy Garcia	<NA>	<NA>	<NA>
	supervisor_district		data_as_of		data_loaded_at
1		1	2026-02-20T16:37:46.000		2026-02-20T16:44:43.000
2		3	2026-02-20T16:37:46.000		2026-02-20T16:44:43.000
3		3	2026-02-20T16:37:46.000		2026-02-20T16:44:43.000
4		<NA>	2026-02-20T16:37:46.000		2026-02-20T16:44:43.000
5		2	2026-02-20T16:37:46.000		2026-02-20T16:44:43.000
6		<NA>	2026-02-20T16:37:46.000		2026-02-20T16:44:43.000
	:@computed_region_6qbp_sg9q :@computed_region_ajp5_b2md				
1		7			17
2		18			23
3		16			21
4		77			<NA>
5		17			13
6		<NA>			<NA>
	:@computed_region_26cr_cadq		actor_3	fun_facts	point.type
1		4	<NA>	<NA>	Point
2		3	Ethan Suplee	<NA>	Point
3		3	Clifton Collins Jr.	<NA>	Point
4		<NA>	Shen-Hao Wen	<NA>	Point
5		6	Peter Coyote	<NA>	Point
6		<NA>	Raven-Symone	<NA>	<NA>

```

    point.coordinates
1 -122.49624, 37.78578
2 -122.40582, 37.80239
3 -122.41103, 37.79278
4 -122.39603, 37.79807
5 -122.44842, 37.80293
6                NULL

```

Interpretation

- GET() sends an HTTP request
- We receive a response object with status code (200 - understood, 404 - not found, etc.) headers and content (the JSON)
- fromJSON() converts the response to a data frame
- They store the larger data, we access it in a legal orderly fashion.

Bigger Challenges - Multiple files

- [Yelp Open Data](#)
- [LA City Parking Meters](#)
- [and their locations](#)
- but really, for next time

For your consideration

- This is a simplified version pre-pandemic

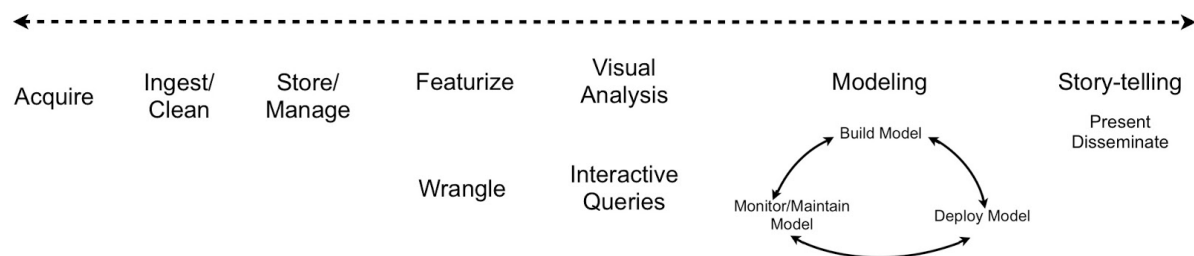


Figure 5: Old Data Science Workflow

Takeaways

- Rectangular format of csv, tsv, txt, Excel is older
- There are even older forms of data that are seen in specific disciplines
- JSON is transitional and is used in many places
- Newer formats (e.g., parquet, arrow, etc.) exist for size & speed and for non-rectangular forms (later)
- But data is just data, don't let the different formats scare/stop you

Thank You